



TRƯỜNG ĐẠI HỌC KINH TẾ KỸ THUẬT CÔNG NGHIỆP
Khoa Công Nghệ Thông Tin



LẬP TRÌNH DI ĐỘNG

CHƯƠNG 4: LẬP TRÌNH USER INTERFACE NÂNG CAO





Chương 4 – Lập trình User Interface nâng cao



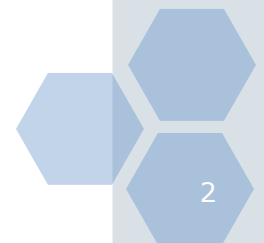
MENU VÀ CÁCH SỬ DỤNG MENU



CÁC THÀNH PHẦN GIAO DIỆN NÂNG CAO



XỬ LÝ SỰ KIỆN





UNETI



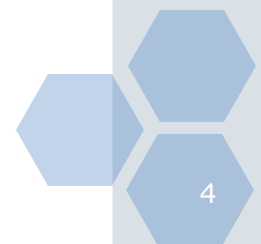
1.1 MENU VÀ CÁCH SỬ DỤNG MENU





1.1 MENU VÀ CÁCH SỬ DỤNG

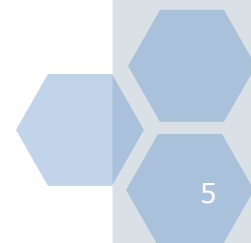
- Menu là một thành phần giao diện người dùng phổ biến trong nhiều loại ứng dụng. Để cung cấp một trải nghiệm người dùng quen thuộc và nhất quán, bạn nên sử dụng các API Menu để trình bày hành động người dùng và các tùy chọn khác trong hoạt động của mình.
- Menu trong Android có 2 loại menu chính là : **OptionsMenu** và **ContextMenu**.



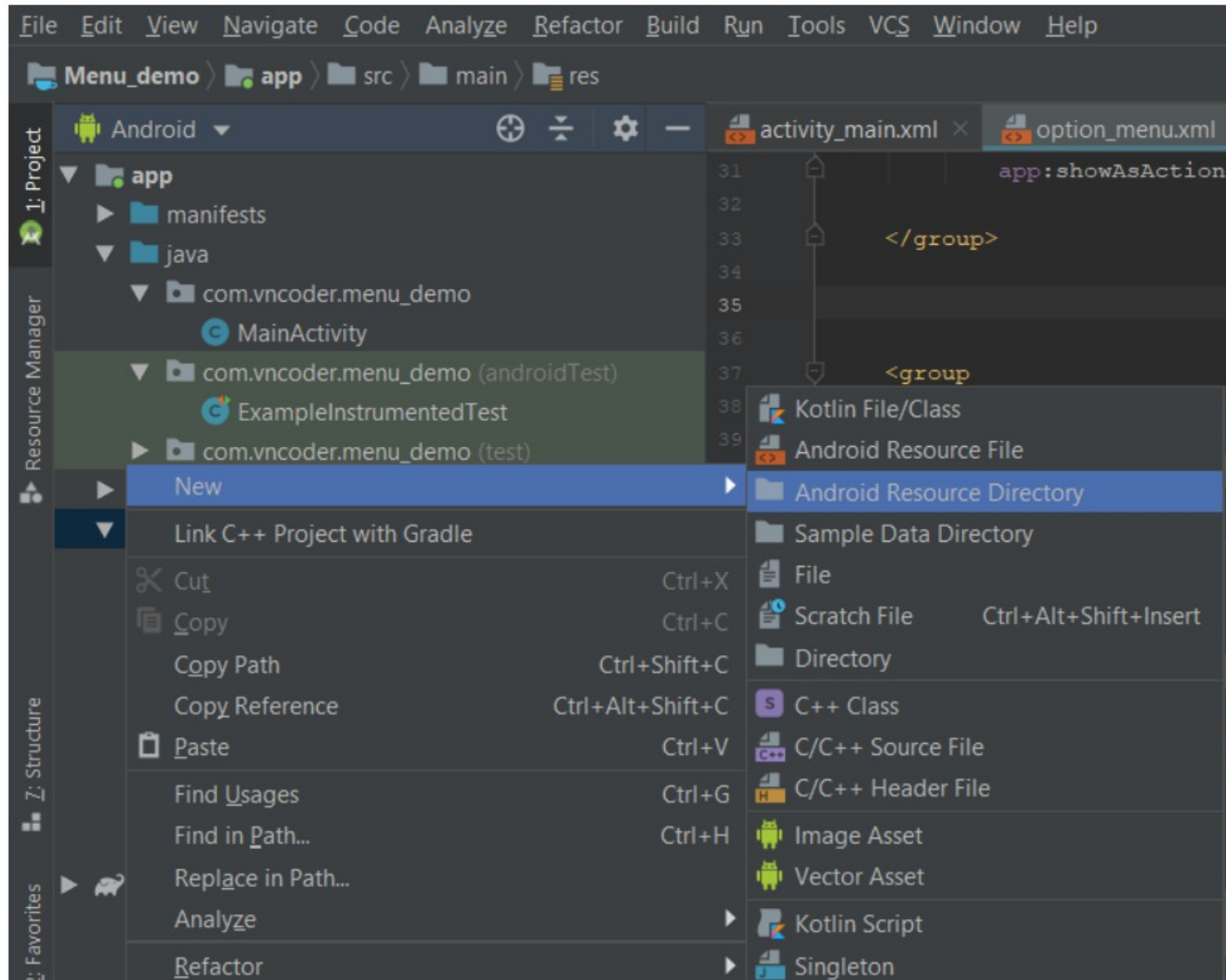


1.1.1 OPTION MENU

- Option Menu là loại menu nằm trên ActionBar hoặc Toolbar ở phía bên phải. Menu hiển thị dưới dạng text, text+icon hoặc icon “ba chấm”, menu cũng hỗ trợ tùy biến giao diện.
- Để tạo menu đầu tiên bạn vào click chuột phải vào mục **res - New - Android Resource Directory** để tạo một thư mục chứa menu và lưu ý là bạn phải đặt chính xác tên thư mục là menu.

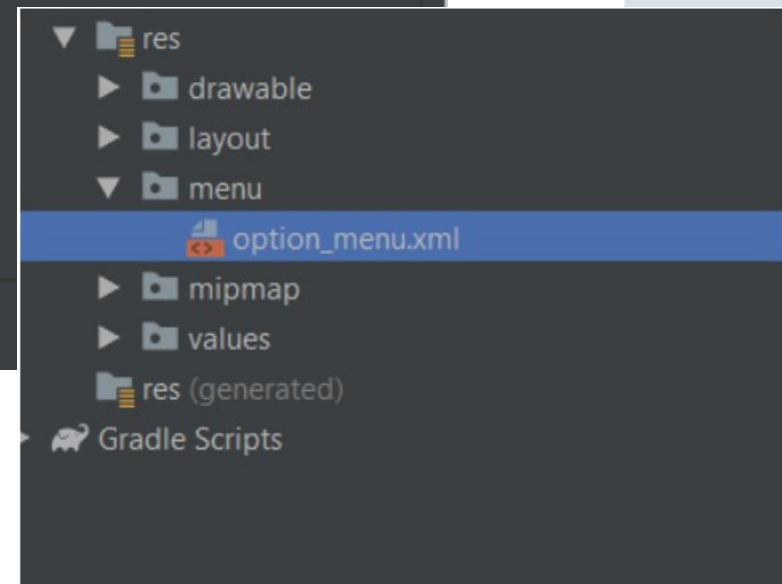
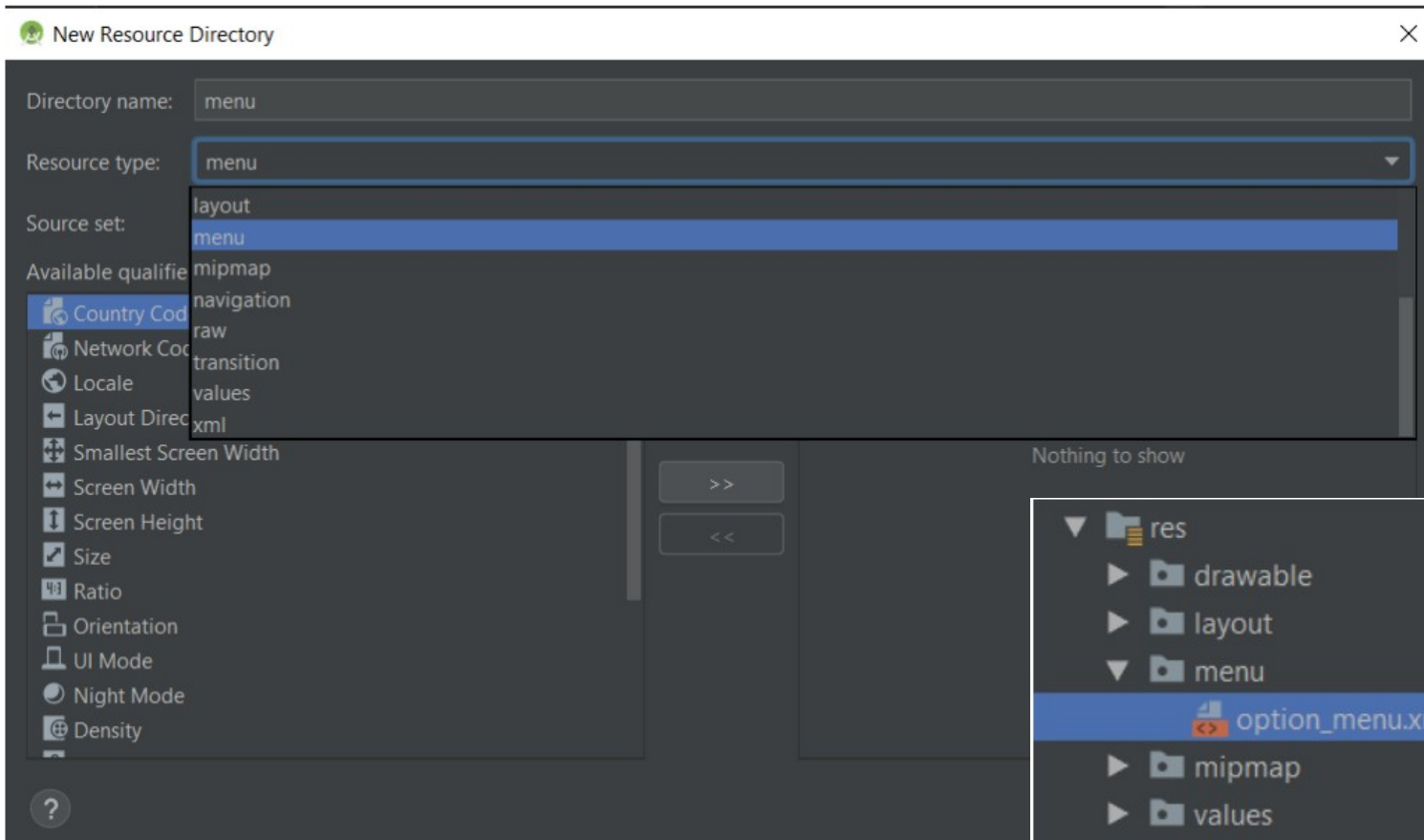


1.1.1 OPTION MENU





1.1.1 OPTION MENU



- Sau khi hoàn thành ta có thư mục menu chứa file **option_menu.xml**



1.1.1 OPTION MENU

- Để tạo các ra menu ta dùng các thẻ `<item>`
- Trong thẻ item có những thuộc tính ta cần lưu ý :
 - **id** : dùng để tham chiếu tới item
 - **title** : tiêu đề của item
 - **icon** : hiển thị icon

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

    <item android:id="@+id/item1"
          android:title="Item1"
          android:icon="@drawable/ic_search_black_24dp"
          android:orderInCategory="100"
          app:showAsAction="ifRoom|collapseActionView"
          app:actionViewClass="android.widget.SearchView"
          />

    <item android:id="@+id/item2"
          android:title="Item2"
          android:icon="@drawable/ic_people_black_24dp"
          android:orderInCategory="100"
          app:showAsAction="withText" />

    <item android:id="@+id/item3"
          android:title="Item3"
          android:icon="@drawable/ic_settings_black_24dp"
          android:orderInCategory="100"
          app:showAsAction="withText" />

    <item android:id="@+id/item4"
          android:title="Item4"
          android:icon="@drawable/ic_photo_camera_black_24dp"
          android:orderInCategory="100"
          app:showAsAction="withText" />

</menu>
```




1.1.1 OPTION MENU

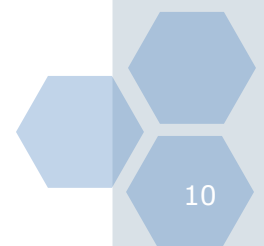
- Trong **MainActivity.java** ta tiến hành Override hàm `onCreateOptionsMenu` để tham chiếu tới menu
- Gọi hàm `getMenuInflater()` để tạo một `MenuInflater` (`MenuInflater` không thể khởi tạo với "new `MenuInflater()`" mà dùng hàm `getMenuInflater()`).

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.option_menu, menu);
    MenuItem menuItem = menu.findItem(R.id.item1);
    SearchView searchView = (SearchView) MenuItemCompat.getActionView(menuItem);
    return super.onCreateOptionsMenu(menu);
}
```



1.1.1 OPTION MENU

- Trong **MainActivity.java** ta Override hàm `onCreateOptionsMenu` để tham chiếu tới menu.
- Gọi hàm `getMenuInflater()` để tạo một `MenuInflater` (khởi tạo qua hàm `getMenuInflater()`).
- `MenuInflater().inflate(R.menu.mymenu,menu)`. Trong hàm `inflate()` tham số thứ nhất là menu cần lấy ra để tạo, tham số thứ 2 là `OptionsMenu`.
- Để gán sự kiện cho các item trong menu ta tiến hành Override hàm `onOptionsItemSelected`.



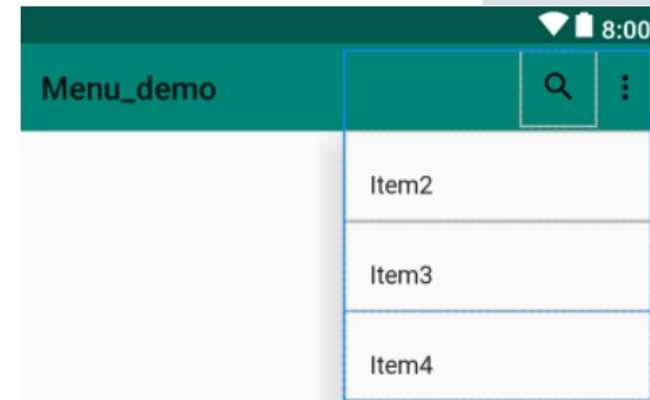
1.1.1 OPTION MENU

@Override

```
public boolean onCreateOptionsMenu(Menu menu) {  
    getMenuInflater().inflate(R.menu.option_menu, menu);  
    MenuItem menuItem = menu.findItem(R.id.item1);  
    SearchView searchView = (SearchView) MenuItemCompat.getActionView(menuItem);  
    return super.onCreateOptionsMenu(menu);  
}
```

@Override

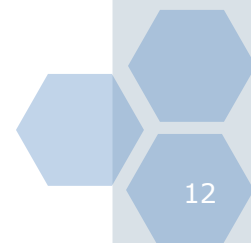
```
public boolean onOptionsItemSelected(@NonNull MenuItem item) {  
    int id = item.getItemId();  
    switch (id) {  
        case R.id.item2:  
            Toast.makeText(getBaseContext(), "item2", Toast.LENGTH_LONG).show();  
            break;  
        case R.id.item3:  
            Toast.makeText(getBaseContext(), "item3", Toast.LENGTH_LONG).show();  
            break;  
        case R.id.item4:  
            Toast.makeText(getBaseContext(), "item4", Toast.LENGTH_LONG).show();  
            break;  
        case R.id.item5:  
            Toast.makeText(getBaseContext(), "item5", Toast.LENGTH_LONG).show();  
            break;  
        case R.id.item6:  
            Toast.makeText(getBaseContext(), "item6", Toast.LENGTH_LONG).show();  
            break;  
        case R.id.item6_1:  
            Toast.makeText(getBaseContext(), "item6.1", Toast.LENGTH_LONG).show();  
        case R.id.item6_2:  
            Toast.makeText(getBaseContext(), "item6.2", Toast.LENGTH_LONG).show();  
        default:  
    }  
    return super.onOptionsItemSelected(item);  
}
```





1.1.2 CONTEXT MENU

- **Context menu** xuất hiện khi người dùng tác động lên một view nào đó. Chẳng hạn khi người dùng click vào Listview, một vài thao tác được hiển thị để người dùng lựa chọn (context menu).



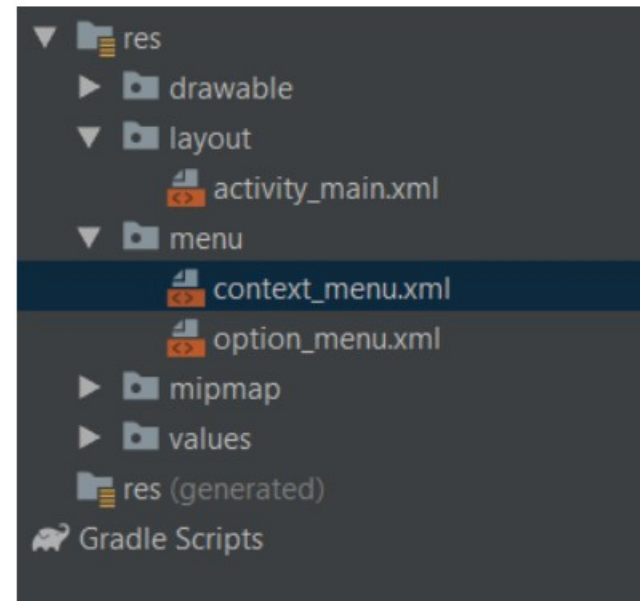
1.1.2 CONTEXT MENU

- Giống như `OptionsMenu` ta cũng cần phải tạo một file menu đặt tên là `context_menu.xml`
- File XML

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
<item android:id="@+id/context1"
      android:title="context menu 1"
      android:icon="@drawable/ic_settings_black_24dp"/>

<item android:id="@+id/context2"
      android:title="context menu 2"
      android:icon="@drawable/ic_settings_black_24dp"/>

</menu>
```



1.1.2 CONTEXT MENU

- Tạo một nút trong MainActivity
- Trong **MainActivity.java** chúng ta thêm 2 hàm để định nghĩa ContextMenu và view của nó.



```
@Override
public void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo) {
    super.onCreateContextMenu(menu, v, menuInfo);
    if (v == btn){
        getMenuInflater().inflate(R.menu.context_menu,menu);
    }
}

@Override
public boolean onContextItemSelected(@NonNull MenuItem item) {
    switch (item.getItemId()){
        case R.id.context1:
            Toast.makeText(getBaseContext(),"context1",Toast.LENGTH_LONG).show();
    }
    return super.onContextItemSelected(item);
}
```



UNETI



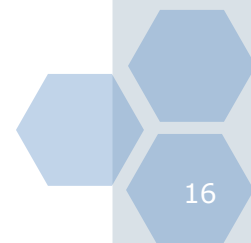
1.2 CÁC THÀNH PHẦN GIAO DIỆN NÂNG CAO





1.2.1 ADAPTER

- Android Adapter (Tạm dịch là bộ tiếp nối) là một cầu nối giữa các View (ví dụ như ListView) và các dữ liệu cơ bản cho View đó.
- Một Adapter có trách nhiệm lấy dữ liệu từ bộ dữ liệu và tạo ra các đối tượng View dựa trên dữ liệu đó
- Bạn có thể tạo các lớp Adapter riêng của bạn từ đầu, nhưng các nhà phát triển khuyên nên sử dụng hoặc thừa kế các lớp Adapter được cung cấp bởi Android SDK, chẳng hạn như ArrayAdapter và SimpleCursorAdapter



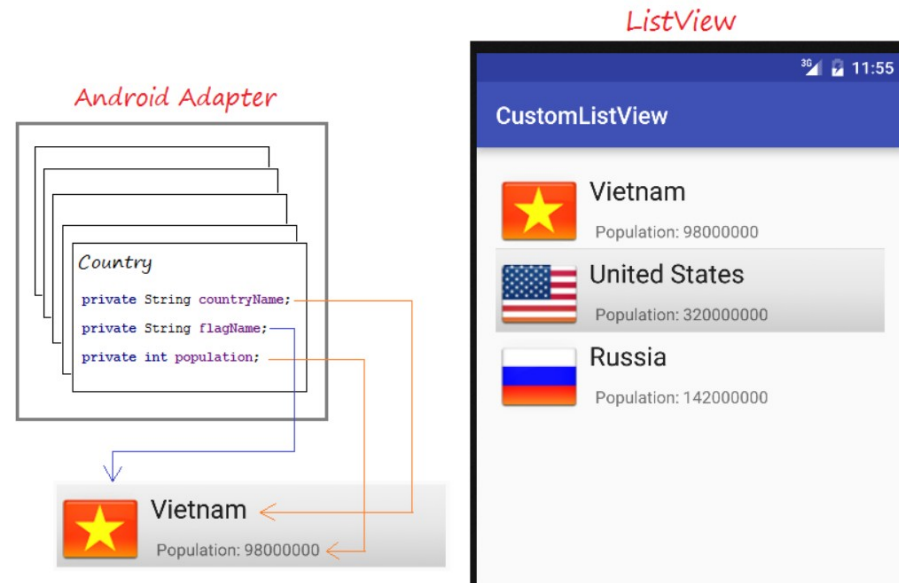


1.2.1 ADAPTER



Example:
Array, Cursor,...

Example:
GridView, ListView,
Spinner, StackView, ...





1.2.1 ADAPTER

Cách tạo Adapter và hiển thị dữ liệu lên ListView

- Để tạo Adapter ta cần: Tập hợp dữ liệu và Layout chứa các đối tượng (View) con được tạo ra

Bước 1: Tạo bộ dữ liệu

- Lớp ArrayAdapter có thể sử dụng cả mảng và các đối tượng List như là bộ dữ liệu

```
String[] cheeses = {  
    "Parmesan",  
    "Ricotta",  
    "Fontina",  
    "Mozzarella",  
    "Cheddar"  
};
```

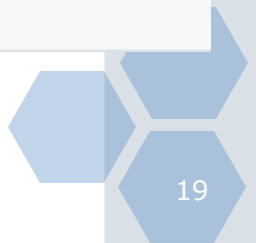


1.2.1 ADAPTER

Bước 2: Tạo tập tin nguồn

- Tạo một file Layout XML có tên **item.xml**, đây là thành phần của ListView

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="https://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="@dimen/activity_horizontal_margin">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textAppearance="?android:attr/textAppearanceLarge"
        android:text="Large Text"
        android:id="@+id/cheese_name" />
</LinearLayout>
```



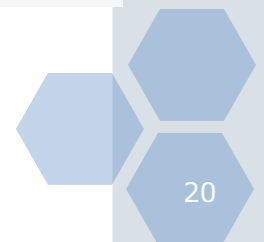


1.2.1 ADAPTER

Bước 3: Tạo Adapter

- Trong Activity của bạn, tạo ra một đối tượng mới của lớp ArrayAdapter

```
ArrayAdapter<String> cheeseAdapter =  
    new ArrayAdapter<String>(this,  
        R.layout.item,  
        R.id.cheese_name,  
        cheeses  
    );
```





1.2.1 ADAPTER

Bước 3: Tạo List

- Trong Activity khởi tạo ListView hoặc kéo thả trong file xml

```
ListView cheeseList = new ListView(this);
```

- Đẩy ListView lên màn hình

```
setContentView(cheeseList);
```

- Liên kết Adapter với ListView

```
cheeseList.setAdapter(cheeseAdapter);
```





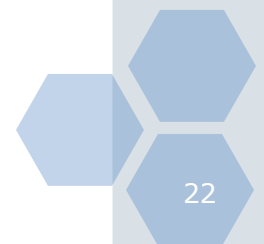
1.2.2 ACTIONBAR

- Mọi ứng dụng, nếu không có thiết kế gì đặc biệt sẽ có ActionBar như sau



SimpleApp

- Tiêu đề của ActionBar được điều khiển bởi file `AndroidManifest.xml`, trong Activity gán tiêu đề vào thuộc tính `android:label`
- Nếu Activity không có thiết lập `android:label` thì nó lấy giá trị mặc định từ `android:label` của application





1.2.2 ACTIONBAR

```
<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/AppTheme">
    <activity
        android:name=".MainActivity"
        android:label="@string/myactivityname" >
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
```

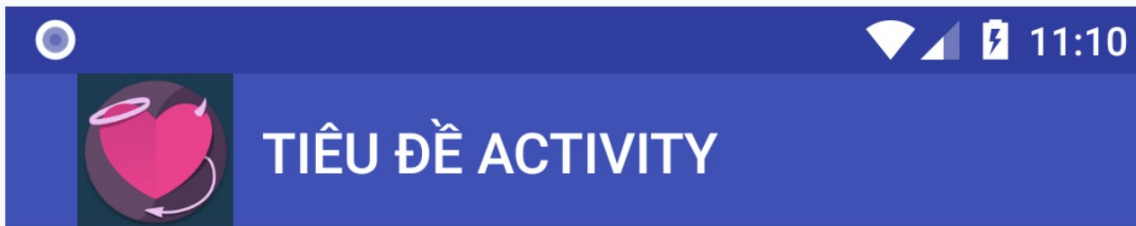
1.2.2 ACTIONBAR

- **Chỉnh tiêu đề bởi code Java**

```
ActionBar actionBar = getSupportActionBar();  
actionBar.setTitle("TIÊU ĐỀ ACTIVITY"); //Thiết lập tiêu đề nếu muốn  
String title = actionBar.getTitle().toString(); //Lấy tiêu đề nếu muốn  
actionBar.hide(); //Ẩn ActionBar nếu muốn
```

- **Thêm Icon vào ActionBar**

```
ActionBar actionBar = getSupportActionBar();  
  
actionBar.setDisplayHomeAsUpEnabled(true);  
actionBar.setLogo(R.mipmap.ic_launcher); //Icon muốn hiện thị  
actionBar.setDisplayUseLogoEnabled(true);
```





1.2.2 ACTIONBAR

- Thêm icon Back vào ActionBar

```
getSupportActionBar().setDisplayHomeAsUpEnabled(true);
```



- Bắt sự kiện khi click nút icon Back: Ta Override phương thức `onOptionsItemSelected(MenuItem item)`. Item là đối tượng biểu diễn nút bấm

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    switch (item.getItemId())
    {
        case android.R.id.home:
            onBackPressed();
            return true;

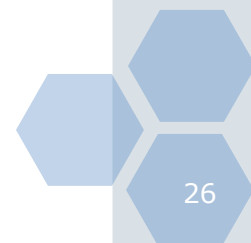
        default:break;
    }

    return super.onOptionsItemSelected(item);
}
```



1.2.3 WEB VIEW

- Là view hiển thị nội dung HTML, thực hiện mã js và một số loại dữ liệu online
- XML file:
`<WebView`
`xmlns:android="http://schemas.android.com/apk/res/android"`
`android:id="@+id/webview"`
`android:layout_width="fill_parent"`
`android:layout_height="fill_parent" />`
- Cấp quyền sử dụng Internet trong manifest file
`<uses-permission android:name="android.permission.INTERNET" />`

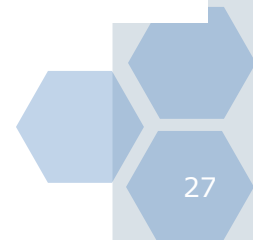




1.2.3 WEB VIEW

```
public class WebActivity extends Activity{
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.webview);

        WebView wv = (WebView) findViewById(R.id.webview1);
        wv.loadUrl("http://mobipro.vn");
    }
}
```





1.2.3 WEB VIEW

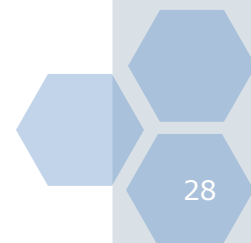
```
WebView wv = (WebView) findViewById(R.id.webview1);

final String mimeType = "text/html";
final String encoding = "UTF-8";
String html = "<H1>Đây là một trang web đơn giản</H1><body>" +
"<p>Bạn có thể tạo ra nội dung HTML bằng cách này</p>";
```

Và nạp lên WebView như sau:

```
wv.loadDataWithBaseURL("", html, mimeType, encoding, "");

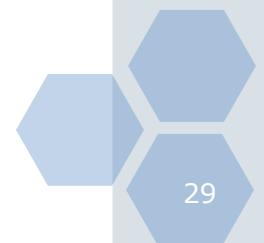
WebView wv = (WebView) findViewById(R.id.webview1);
wv.loadUrl("file:///android_asset/Index.html");
```





1.2.3 WEB VIEW

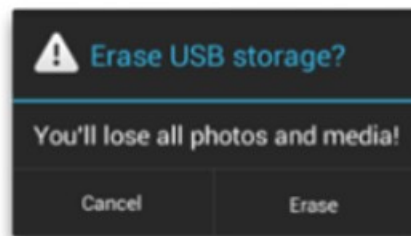
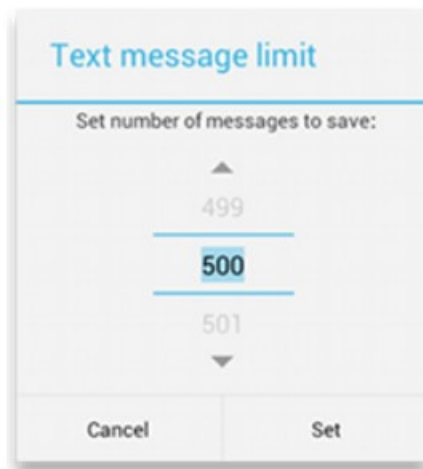
- Bật hỗ trợ javascript:
`myWebView.getSettings().setJavaScriptEnabled(true);`
- Có thể từ javascript gọi các method của ứng dụng: Xem thêm **addJavascriptInterface**
- WebView bản thân nó chỉ là một view để hiển thị trang web, không phải là browser, vì thế khi tác động vào trang web, android sẽ xử lý mặc định, muốn xử lý việc này xem thêm **WebViewClient**
- Cần chú ý một số phím mặc định trong trường hợp WebView được focus (chẳng hạn phím back)





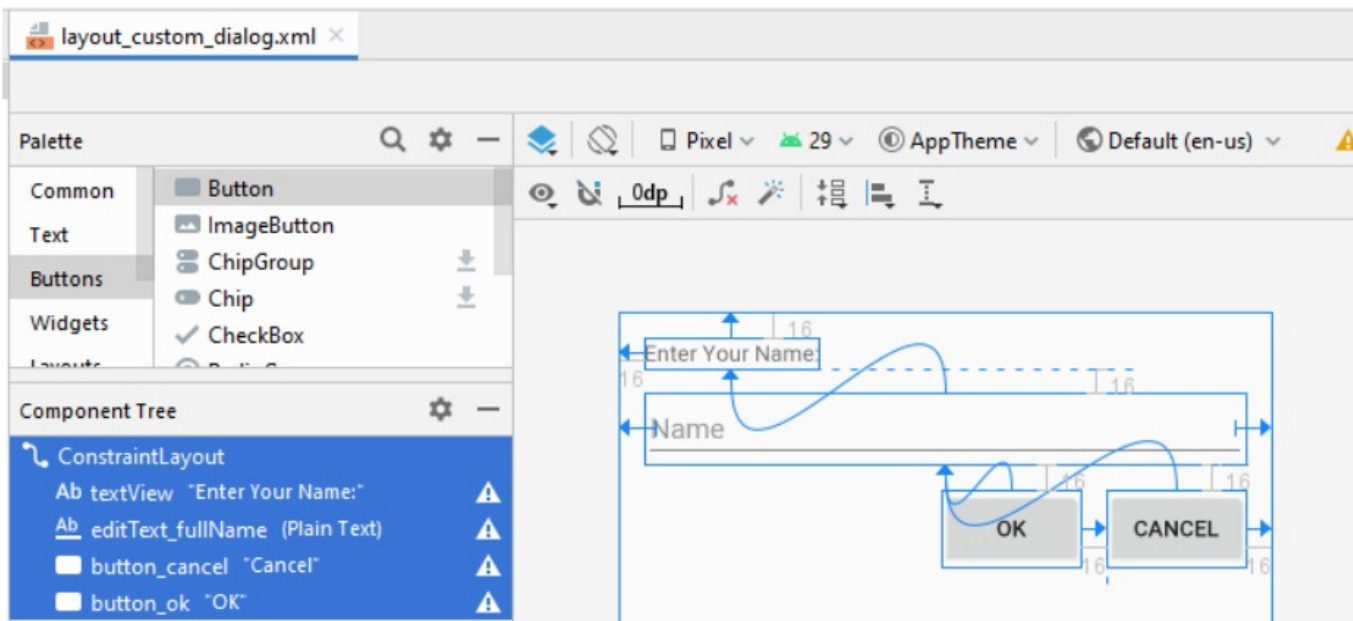
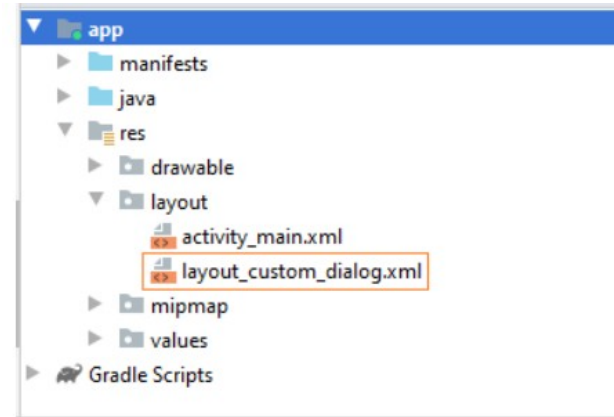
1.2.4 DIALOG

- Trong Android, Dialog (Hộp thoại) là một cửa sổ nhỏ xuất hiện để nhắc người dùng đưa ra một quyết định hoặc nhập vào các thông tin bổ sung.
- Dialog không lấp kín toàn bộ màn hình, và nó thường hiển thị trong chế độ modal, người dùng phải đưa ra quyết định để đóng nó lại thì mới có thể tương tác với các phần khác của ứng dụng.



1.2.4 DIALOG

- Tạo file Layout Dialog như sau:





1.2.4 DIALOG

- Tạo lớp CustomDialog kế thừa từ lớp Dialog

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    requestWindowFeature(Window.FEATURE_NO_TITLE);
    setContentView(R.layout.layout_custom_dialog);

    this.editTextFullName = (EditText) findViewById(R.id.editText_fullName);
    this.buttonOK = (Button) findViewById(R.id.button_ok);
    this.buttonCancel = (Button) findViewById(R.id.button_cancel);

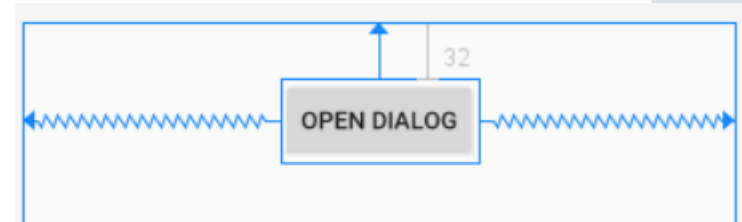
    this.buttonOK.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            buttonOKClick();
        }
    });
    this.buttonCancel.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            buttonCancelClick();
        }
    });
}
```




1.2.4 DIALOG

- Tạo một nút trong activity layout
- Trong MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
  
    private Button buttonOpenDialog;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        this.buttonOpenDialog = (Button) this.findViewById(R.id.button_openDialog);  
  
        this.buttonOpenDialog.setOnClickListener(new View.OnClickListener() {  
            @Override  
            public void onClick(View v) {  
                buttonOpenDialogClicked();  
            }  
        });  
    }  
  
    private void buttonOpenDialogClicked() {  
        CustomDialog.FullNameListener listener = new CustomDialog.FullNameListener() {  
            @Override  
            public void fullNameEntered(String fullName) {  
                Toast.makeText(MainActivity.this, "Full name: " + fullName, Toast.LENGTH_LONG).show();  
            }  
        };  
        final CustomDialog dialog = new CustomDialog(this, listener);  
        dialog.show();  
    }  
}
```





UNETI



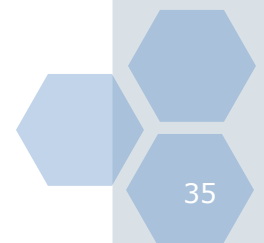
1.3 XỬ LÝ SỰ KIỆN





1.3.1 SỰ KIỆN TRONG ANDROID

- Sự kiện được tạo ra để đáp ứng với một hành động do tác động từ bên ngoài, thường là tương tác bởi người dùng
- Có 4 cách chính để cài đặt sự kiện trong lập trình Android như:
 - Cài đặt hàm xử lý trong class Activity và khai báo sử dụng ngoài layout
 - Cài đặt động thông qua các hàm setListener trong class Activity
 - Implements Listener trong khai báo lớp
 - Dùng biến lưu trữ





1.3.2 CÁCH 1: Onclick in XML

**TRONG FILE
CODE JAVA**

```
public void OnButtonClick(View v)
{
    TextView tv1=(TextView)findViewById(R.id.textview1);
    tv1.setText("Click 1");
}
```

```
<Button
    android:id="@+id/button1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Click 1"
    android:onClick="OnButtonClick"
/>
```

**TRONG FILE
LAYOUT XML**



1.3.3 CÁCH 2: Inline anonymous listener

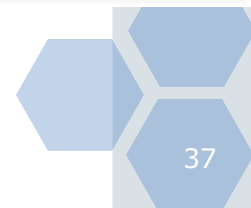
Sự kiện đó sẽ được cài đặt khi Activity chứa View đó được tạo ra, điều đó cũng có nghĩa mình sẽ cài việc đó vào hàm onCreate của Activity đó.

```
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);

    Button b1=(Button)findViewById(R.id.button1); //tìm Button mang id là
button1
    b1.setOnClickListener(new OnClickListener() {

        @Override
        public void onClick(View v) {
            TextView tv1=(TextView)findViewById(R.id.textview1);
            tv1.setText("Click 1");
        }

    });
}
```





1.3.4 CÁCH 3: Activity is listener

```
public class ClickActivity extends Activity implements OnClickListener {

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        Button b1=(Button)findViewById(R.id.button1);
        b1.setOnClickListener(this);
    }

    @Override
    public void onClick(View v) {
        // TODO Auto-generated method stub
        TextView tv1=(TextView)findViewById(R.id.textview1);
        tv1.setText("Click 1");
    }
}
```




1.3.5 CÁCH 4: Listener in variable

```
public class ClickActivity extends Activity {

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        Button b1=(Button)findViewById(R.id.button1);
        b1.setOnClickListener(buttonClickListener);
    }

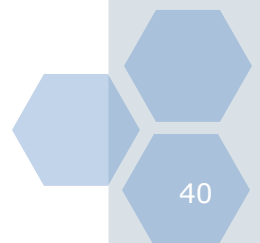
    OnClickListener buttonClickListener = new OnClickListener() {
        @Override
        public void onClick(View v) {
            TextView tv1=(TextView)findViewById(R.id.textview1);
            tv1.setText("Click 1");
        }
    };
}
```



1.3.6 MỘT SỐ CÁCH ÍT THÔNG DỤNG HƠN

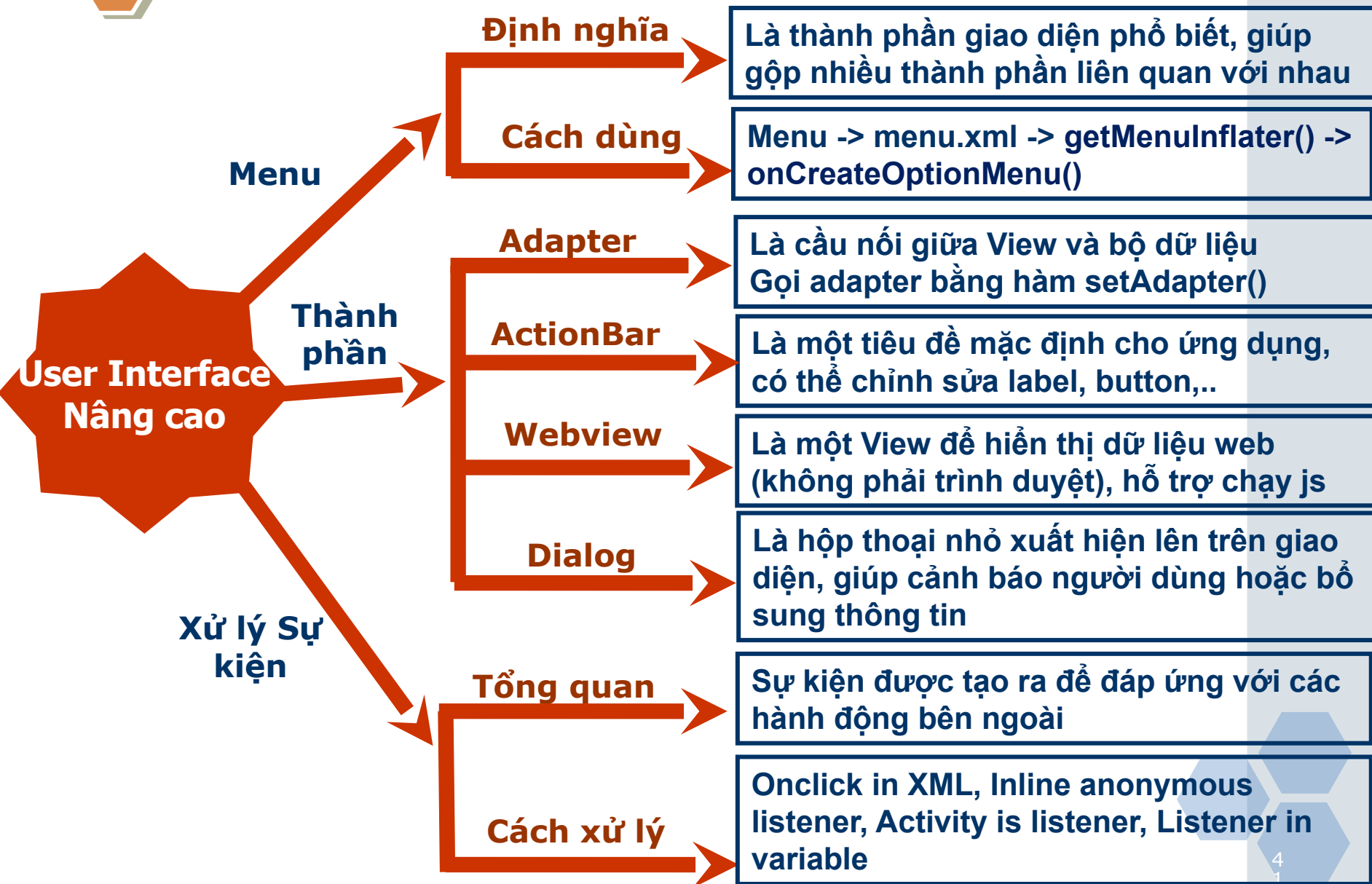
- **Cách 5:** Explicit listener class
 - Viết class độc lập xử lý sự kiện
 - Class có thể là inner class hoặc hoàn toàn độc lập
- **Cách 6:** Anonymous view subclassing

```
layout.addView(new Button(this) {  
    @Override  
    public boolean performClick() {  
        // đoạn mã xử lý nút bấm  
        ...  
        return false;  
    }  
});
```





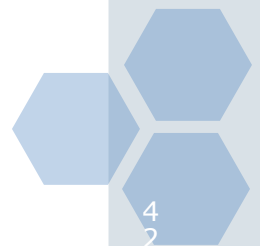
Tổng kết





Câu hỏi lý thuyết

1. Adapter là gì? Ngoài ListView, Adapter có thể tương tác với View khác không?
2. Nêu các loại Dialog khác mà bạn biết
3. Chỉ ra ưu, nhược điểm của các cách xử lý sự kiện
4. Trong những cách xử lý sự kiện trên, bạn thích cách nào nhất? Lý do?





Bài tập

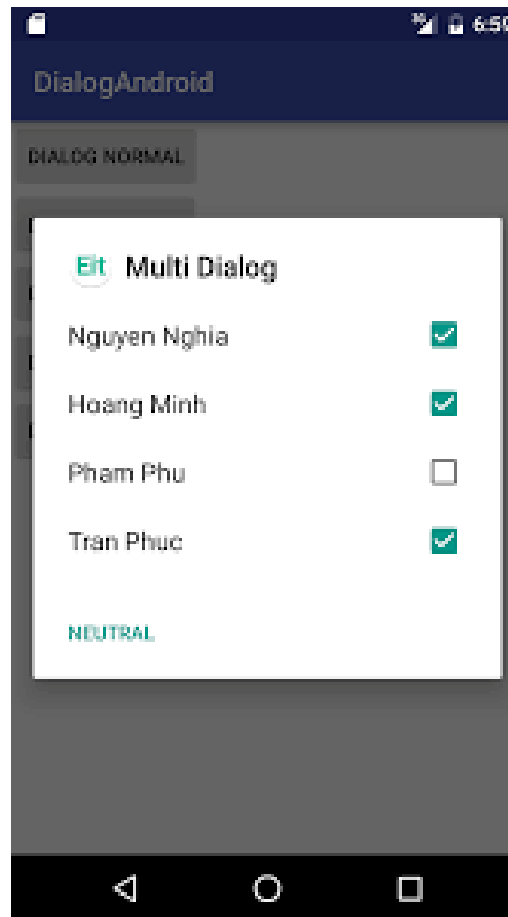
1. Sử dụng ListView và Adapter, xây dựng giao diện sau





Bài tập

2. Sử dụng Dialog xây dựng giao diện sau





Bài tập

3. Sử dụng webview, xây dựng giao diện sau

